APPLICATION-BASED BENCHMARKING

Redis and MongoDB for Trip Planning Using GTFS Data

Software Engineering Checkpoint

Prepared by
Taher Amine ELHOUARI

2026

Benchmarking scope
This checkpoint compares Redis and MongoDB for GTFS trip-planning workloads using application-relevant criteria. No synthetic performance figures are presented as measured results unless they have actually been collected in a controlled test.

1. Introduction to GTFS Data

The General Transit Feed Specification (GTFS) is a standard format used by public transportation agencies to publish information about their transportation networks. GTFS is commonly divided into GTFS Schedule, which contains planned/static transportation information, and GTFS Realtime, which can provide live information such as vehicle positions, delays, trip updates, and service alerts.

A typical GTFS Schedule dataset contains several text files, including:

- stops.txt - stations and stops with latitude and longitude.
- routes.txt - bus, train, metro, or other transit routes.
- trips.txt - individual trips belonging to routes.
- stop_times.txt - arrival and departure times for each stop in a trip.
- calendar.txt and calendar_dates.txt - dates on which services operate.
- shapes.txt - geographical paths followed by vehicles.
- agency.txt - information about the transportation operator.

For trip planning, the application can combine this information to answer questions such as:

- Which stops are closest to the passenger?
- Which buses or trains leave after a particular time?
- Which route connects an origin and destination?
- What transfers are required?
- What is the expected arrival time?

For example, if a passenger wants to travel from Stop A to Stop D, the application can search stop_times, trips, and routes to identify possible journeys and calculate an appropriate itinerary.

2. Overview of Redis and MongoDB

Redis

Redis is a high-performance database built around key-based access and specialized data structures. It supports structures including strings, hashes, lists, sets, sorted sets, streams, and geospatial indexes.

Redis is particularly interesting for trip planning because its geospatial functionality can store coordinates and efficiently search for locations within a radius or geographical area.

GTFS stops could therefore be stored in Redis using a command conceptually similar to:

```
GEOADD gtfs:stops longitude latitude stop_id
```

Information about an individual stop could then be stored in a hash:

```
HSET stop:1001 name "Central Station" route "R1"
```

Redis sorted sets can also associate elements with numeric scores, which makes them useful for time-ordered information such as departure times.

Redis is commonly used when very low-latency access is important. Although it is often associated with memory-based workloads, it can also provide persistence through mechanisms such as RDB snapshots and Append Only Files (AOF).

MongoDB

MongoDB is a document-oriented NoSQL database. Instead of storing information in relational tables and rows, MongoDB stores records as BSON documents, which are similar to JSON objects but support additional data types.

For example, a GTFS stop could be represented as:

```
{
  "stop_id": "1001",
  "name": "Central Station",
  "location": {
    "type": "Point",
    "coordinates": [3.0588, 36.7538]
  }
}
```

MongoDB provides flexible data modeling, allowing documents in a collection to contain different fields when necessary. Data that is frequently accessed together can also be embedded in the same document to reduce the need for joins.

MongoDB also provides indexes, aggregation operations, and dedicated geospatial indexes. A 2dsphere index, for example, can support proximity calculations and queries involving geographical areas.

Difference from Traditional SQL Databases

A relational SQL database normally organizes GTFS information into normalized tables connected through relationships and joins. Redis instead focuses on keys and specialized data structures, while MongoDB organizes related information as documents and collections. This gives developers greater flexibility, but it also means that the database structure should be designed around the application's expected access patterns.

3. Benchmarking Criteria

To compare Redis and MongoDB fairly for a GTFS trip-planning application, I would evaluate several criteria.

Query Latency

I would measure the execution time for important operations, including:

- Looking up a stop by ID.
- Finding nearby stops.
- Finding upcoming departures.
- Retrieving all stops belonging to a trip.
- Finding trips associated with a route.

The benchmark should measure not only average response time but also values such as median, p95, and p99 latency, because an application may perform well on average while still producing slow responses for some passengers.

Throughput

Throughput measures how many operations the database can process per second. This is important when thousands of passengers access the trip planner simultaneously.

Data Ingestion Speed

GTFS datasets may contain very large stop_times.txt files. Therefore, I would measure how long Redis and MongoDB take to import the complete feed.

Storage and Memory Consumption

I would compare RAM consumption, disk utilization, index overhead, and the total storage required for the same GTFS dataset.

Concurrent Users

A benchmark should gradually increase the number of simulated passengers to determine how the databases behave under load.

Scalability

I would test whether performance remains acceptable when more transit agencies are added, the number of stops increases, the number of trips increases, and concurrent queries increase.

Benchmarking is important because database performance depends heavily on the application's real access patterns. A database that performs extremely well for simple key lookups may not necessarily be the best database for complex filtering or geographic searches.

4. Data Ingestion and Storage

Redis

GTFS files can first be parsed using an application written in languages such as Python, JavaScript, or Java. Each GTFS structure can then be mapped to an appropriate Redis data structure.

```
stop:1001               -> Hash containing stop information
gtfs:stops              -> Geospatial index of all stops
route:R1:trips          -> Set containing trip IDs
trip:T100:departures    -> Sorted set ordered by departure time
trip:T100               -> Hash containing trip information
```

A Redis geospatial index would make nearby-stop searches particularly convenient because Redis directly supports searches around geographical coordinates.

MongoDB

GTFS data could be imported into collections such as:

```
agencies
stops
routes
trips
stop_times
calendars
```

Indexes could then be created on fields frequently used for searches:

```
db.stops.createIndex({ stop_id: 1 })
db.trips.createIndex({ trip_id: 1 })
db.trips.createIndex({ route_id: 1 })
db.stop_times.createIndex({ trip_id: 1, stop_sequence: 1 })
db.stops.createIndex({ location: "2dsphere" })
```

Indexes allow MongoDB to avoid scanning every document when suitable indexed queries are executed. The major storage difference is that Redis represents the application data through keys and purpose-specific structures, whereas MongoDB stores structured BSON documents. MongoDB therefore provides a more natural representation for complex entities such as routes, trips, and stops, while Redis can provide extremely efficient structures for specific access patterns.

5. Query Performance

A trip-planning application performs several types of queries.

Query 1 - Find Stops Near a Passenger

In Redis, a geospatial query can search around the passenger's coordinates:

```
GEOSEARCH gtfs:stops
FROMLOC passenger_location
BYRADIUS 1000 m
```

Redis geospatial indexes are specifically designed for operations such as finding points within a radius or geographical bounding area. In MongoDB, the same problem can be solved using a geospatial query such as $geoNear. MongoDB's $geoNear aggregation stage can order documents according to their proximity to a specified location when an appropriate geospatial index exists.

Query 2 - Find Upcoming Departures

An application might request:

```
Find all departures from stop 1001
between 08:00 and 08:30.
```

In Redis, departure times can be represented using sorted sets, with the departure time converted into a numeric score. In MongoDB, the application can query indexed departure-time fields:

```
db.stop_times.find({
  stop_id: "1001",
  departure_seconds: {
    $gte: 28800,
    $lte: 30600
  }
})
```

Query 3 - Retrieve a Complete Trip

A passenger may select a journey and request:

```
Return every stop for trip T100
in stop sequence order.
```

MongoDB can efficiently perform this type of structured query if the collection has a compound index such as:

```
{ trip_id: 1, stop_sequence: 1 }
```

Redis can also perform the lookup quickly when the information has been deliberately precomputed and stored using an appropriate structure.

Benchmark Interpretation

For highly predictable operations such as retrieving a known key, checking cached results, looking up departure times, or locating nearby objects, Redis is an excellent candidate. MongoDB provides greater flexibility when a query combines several properties or when developers need to manipulate richer transportation documents.

Neither database alone automatically solves the entire public-transport routing problem. The application still needs routing logic to evaluate possible paths, transfers, departure times, and journey costs.

6. Scalability and Efficiency

Both databases can scale beyond a single machine, but they use different approaches. Redis Cluster can distribute keys across multiple Redis nodes, allowing the dataset and workload to be partitioned across a cluster. MongoDB provides sharding, which distributes data across multiple machines and is designed to support very large datasets and high-throughput deployments.

For a very large GTFS system containing information from many cities or countries, MongoDB would be my preferred primary database. The main reasons are:

1. GTFS data contains several related but structurally complex entities.

2. MongoDB provides flexible document modeling.

3. It provides powerful indexes and aggregation operations.

4. It supports geospatial indexing.

5. It can distribute large collections through sharding.

However, I would strongly consider adding Redis alongside MongoDB in a real production architecture. MongoDB could store the complete GTFS dataset, while Redis could cache highly requested results such as:

```
nearest stops
upcoming departures
popular journeys
route summaries
recent search results
```

This would combine MongoDB's flexible persistent storage with Redis's low-latency data-access capabilities.

7. Practical Application

I would design a simple application called GTFS Trip Planner. The application would allow a passenger to:

1. Enter an origin.

2. Enter a destination.

3. Select a departure time.

4. Find nearby transit stops.

5. Identify possible trips.

6. Display the recommended journey.

Proposed Architecture

Implementation Steps

Step 1 - Obtain GTFS Data

Download a GTFS feed from a transportation agency and validate its structure. GTFS provides standardized schedule information that applications can process consistently across different transportation networks.

Step 2 - Parse the Dataset

The backend reads stops.txt, routes.txt, trips.txt, stop_times.txt, and calendar.txt. The CSV information is converted into application objects.

Step 3 - Import the Information

Store the complete feed in MongoDB using collections such as stops, routes, trips, stop_times, and services.

Step 4 - Create Indexes

Create indexes for frequently searched fields, particularly stop_id, trip_id, route_id, departure_time, and location. MongoDB geospatial indexing would be used for stop-location searches.

Step 5 - Find Nearby Stops

When the user provides coordinates, the application searches for nearby origin and destination stops.

Step 6 - Find Candidate Trips

The system retrieves trips serving the origin stop and determines which of those trips can reach the destination either directly or through transfers.

Step 7 - Evaluate Schedules

The application examines departure and arrival times and removes journeys that are impossible based on the requested departure time.

Step 8 - Rank the Results

Possible journeys can be ranked according to earliest arrival, fewest transfers, shortest travel time, and walking distance.

Step 9 - Add Redis Caching

Redis can cache frequently calculated responses. If another passenger requests a similar journey while the cached result is still valid, the API can return it without recalculating the complete route.

```
tripplan:origin:destination:time
```

8. Conclusion

Redis and MongoDB are both suitable NoSQL technologies for parts of a GTFS trip-planning application, but they provide different advantages.

Redis Advantages

- Excellent for low-latency access.
- Powerful key-based access.
- Sorted sets are useful for ordered departure information.
- Built-in geospatial functionality is useful for nearby-stop searches.
- Very useful as an application cache.

Redis Disadvantages

- Complex GTFS relationships require careful key and data-structure design.
- Maintaining many interconnected entities can become more complicated than using documents.
- Memory requirements can become important with very large datasets.
- The application developer must design structures around specific query patterns.

MongoDB Advantages

- Flexible BSON document model.
- Good representation of complex GTFS entities.
- Powerful indexing and querying.
- Geospatial query support.
- Horizontal scaling through sharding.
- Easier to use as the main persistent database for a complete transit dataset.

MongoDB Disadvantages

- For simple cached lookups, it may perform more work than a purpose-designed Redis structure.
- Indexes must be planned carefully for high-volume queries.
- Large collections and complex queries require appropriate schema and shard-key design.

Final Recommendation

If I had to select only one database for a large GTFS trip-planning project, I would choose MongoDB because it offers a better balance between structured storage, query flexibility, geospatial capabilities, persistence, and scalability.

However, my preferred production architecture would use MongoDB and Redis together. MongoDB would act as the main source of GTFS information, while Redis would operate as a high-speed caching and lookup layer for frequently requested information.

```
MongoDB = primary GTFS data store
Redis   = high-performance cache and lookup layer
```

This hybrid architecture provides both the flexibility required to manage complex transportation information and the performance required by a large-scale real-time trip-planning application.

References

GTFS - What is GTFS?: https://gtfs.org/getting-started/what-is-gtfs/

GTFS Schedule Reference: https://gtfs.org/documentation/schedule/reference/

Redis Data Types: https://redis.io/docs/latest/develop/data-types/

Redis Geospatial: https://redis.io/docs/latest/develop/data-types/geospatial/

Redis Sorted Sets: https://redis.io/docs/latest/develop/data-types/sorted-sets/

Redis Persistence: https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/

Redis Cluster / Scaling: https://redis.io/docs/latest/operate/oss_and_stack/management/scaling/

MongoDB Documents: https://www.mongodb.com/docs/manual/core/document/

MongoDB Indexes: https://www.mongodb.com/docs/manual/indexes/

MongoDB Geospatial Queries: https://www.mongodb.com/docs/manual/geospatial-queries/

MongoDB 2dsphere Indexes:
https://www.mongodb.com/docs/manual/core/indexes/index-types/geospatial/2dsphere/

MongoDB Sharding: https://www.mongodb.com/docs/manual/sharding/